

Image Processing - Lecture 8



Amir Atapour-Abarghouei

amir.atapour-abarghouei@durham.ac.uk

Reading:

Szeliski - Section 3.4

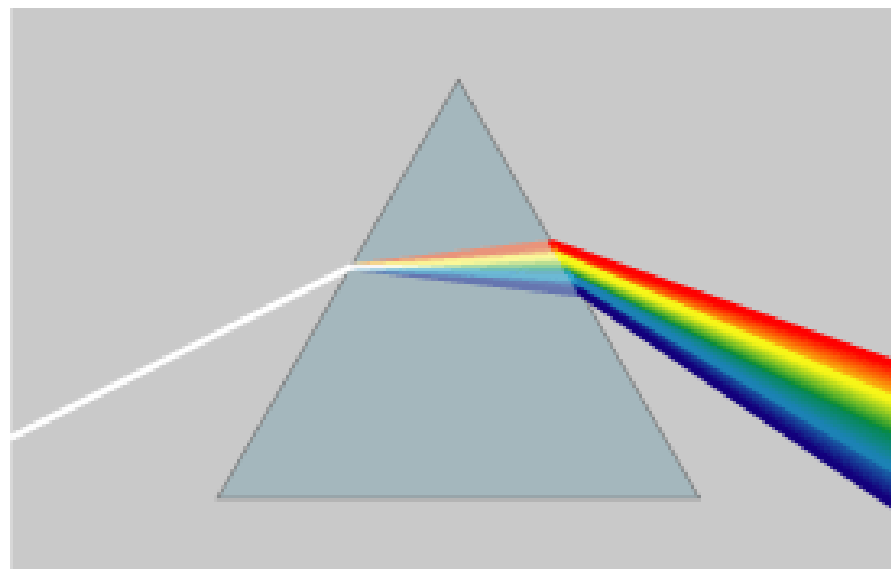
Gonzalez/Woods - Chapter 4

Solomon/Breckon - Chapter 5

Fourier Images in Filtering: Fourier Space II

Slide material acknowledgements: Fisher; Breckon; Ivrisimtzis; Atapour;

Recap - The Fourier Transform



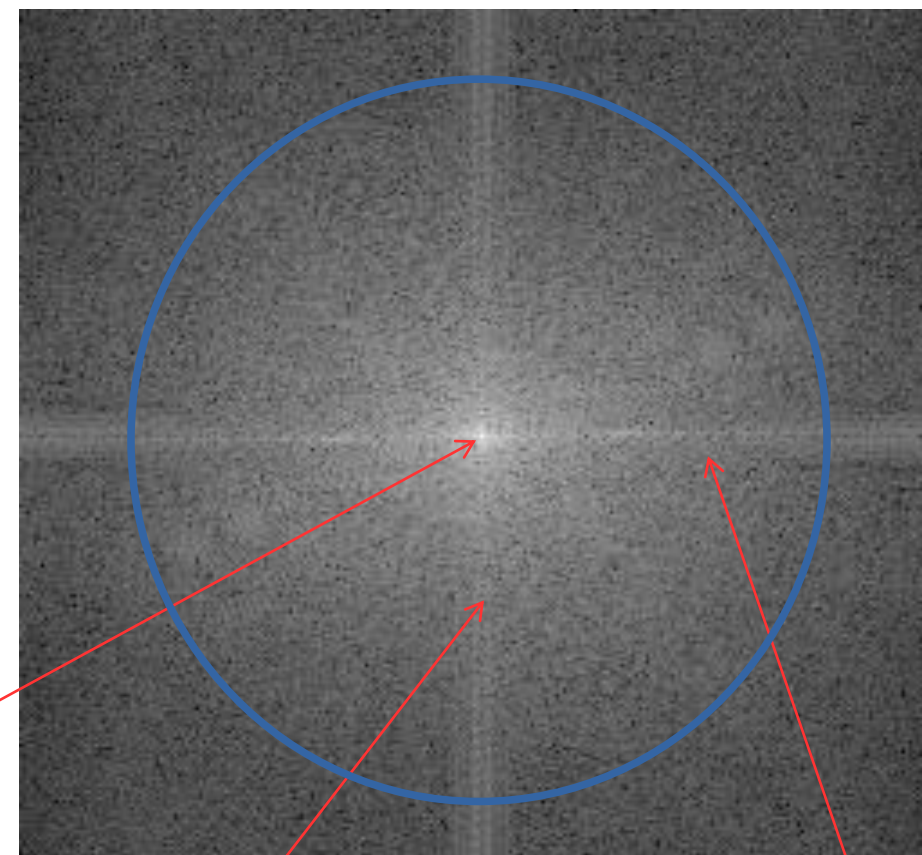
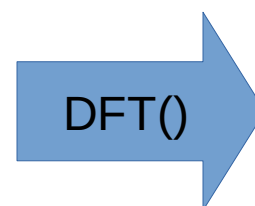
A useful **analogy to the Fourier Transform** is a glass prism.

A glass prism is a physical device that separates light into various colours based on its wavelength (or frequency) content.

The Fourier transform → a ‘**mathematical prism**’ that **separates a function [or image] into components** based on frequency content.

- Gonzalez / Woods 2001

Recap: Transform to Fourier Space



What is

Outside the circle we have _____ frequencies and inside we find _____ frequencies.

Recap: DFT Filtering

What has happened at A ?

What has happened at B ?

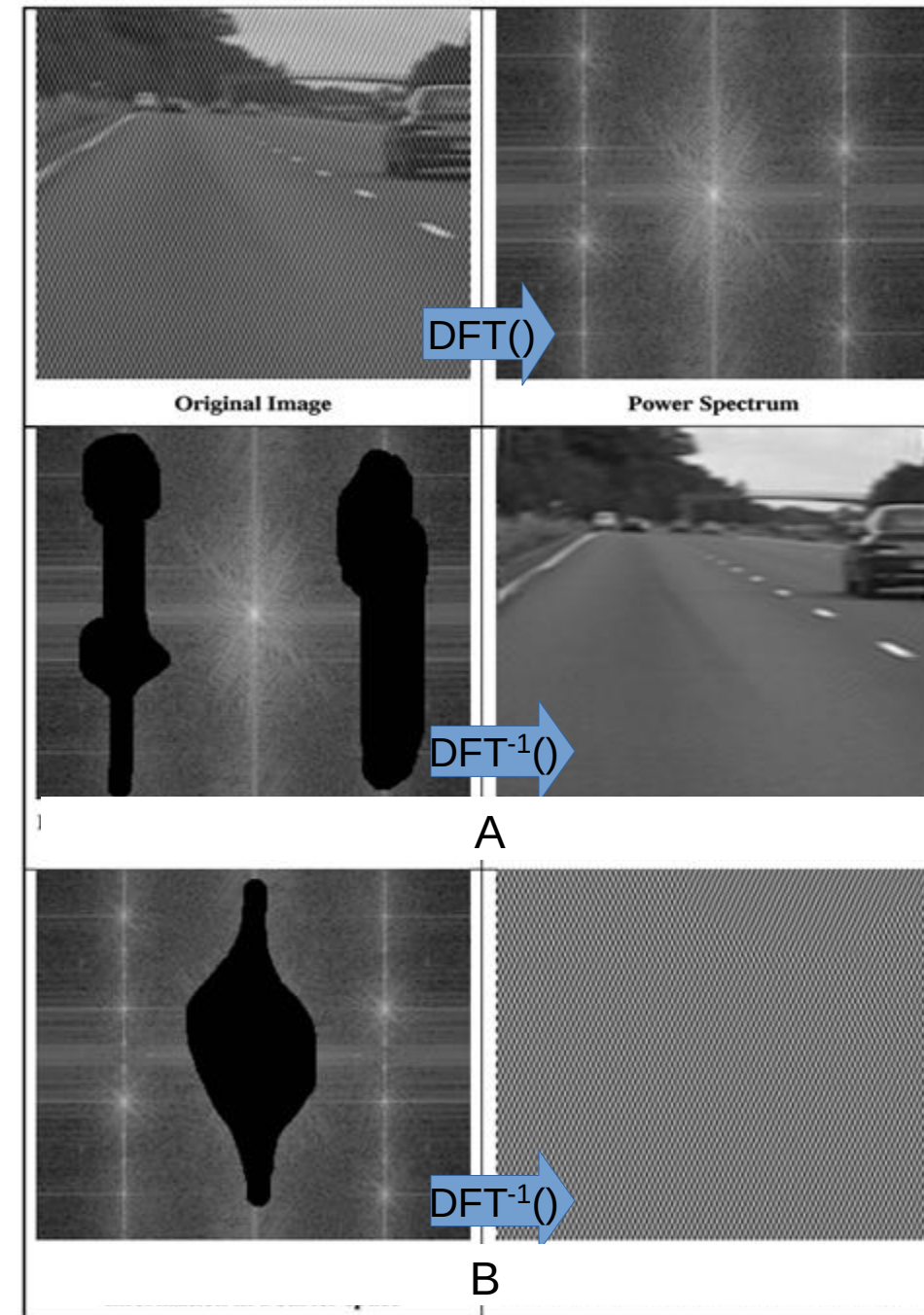


Image Filtering in Fourier Space

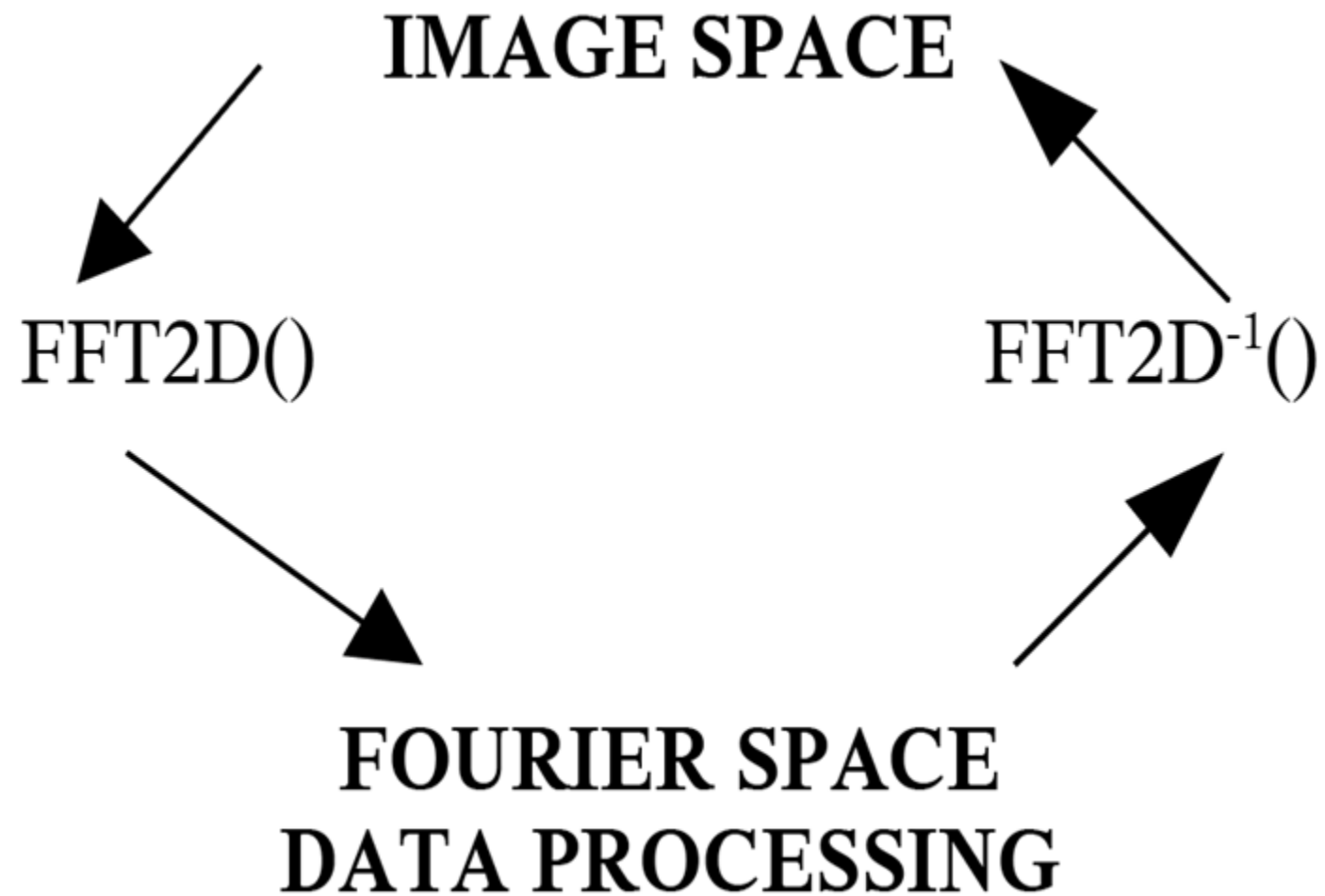


Image Filtering in Fourier Space

Complex operations in the spatial domain can be computed by **simpler** and **computationally more efficient** operations in Fourier space:

- Convolution
- Correlation
- Deconvolution (beyond scope of module)

Simpler operations in **Fourier space**:

- computationally more efficient
- **manipulation** of spatial image content as coefficients **in Fourier representation**

For Example ...

- Recall *Lecture 5*



$$I_{output} = \sum \sum m I_{input}$$

	0	1	0	
	1	-4	1	
	0	1	0	

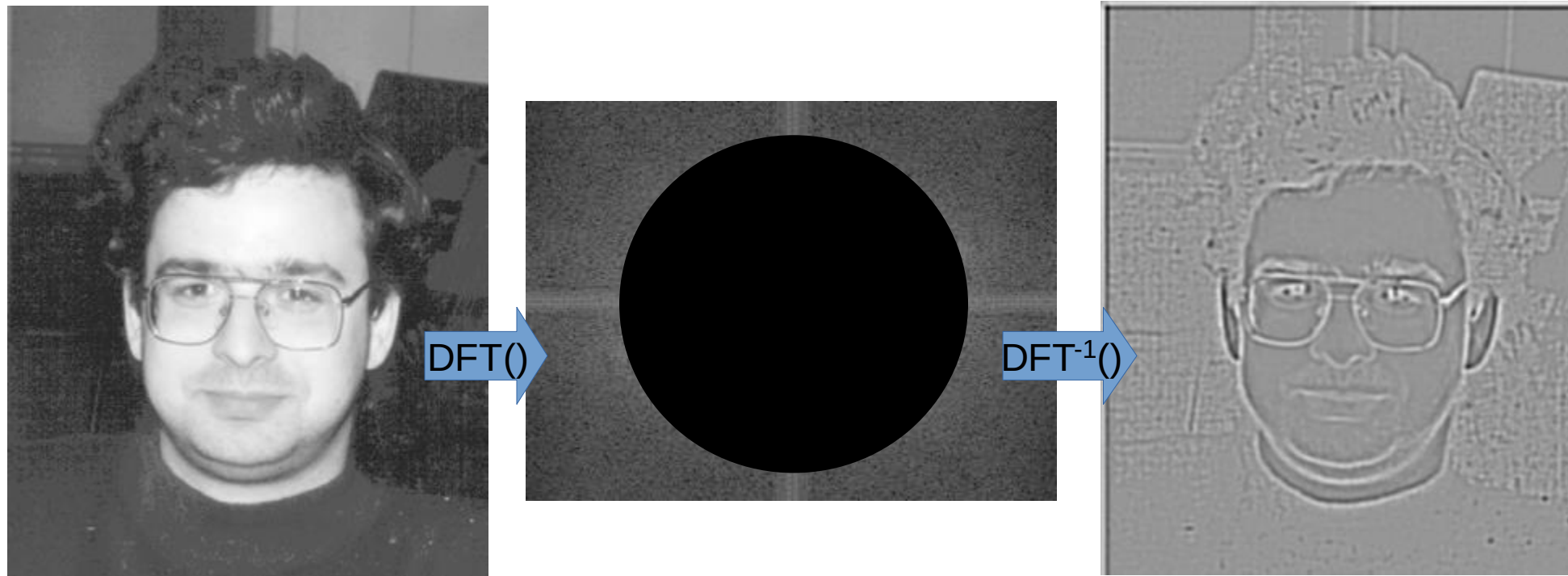


... where we performed spatial convolution (for example with a Laplacian kernel) to find the edges (*high-frequency information*) in the image.

❑ Computationally expensive!

But in Fourier Space

we just turn off all the low-frequency coefficients within the DFT representation



... by multiplying them by 0, to keep the high frequency edges.

Discrete Convolution Theorem

- **Multiplication** in the **frequency domain** reduces to **convolution** in the **spatial domain**.

$$f_{\text{output}} = \sum \sum m I_{\text{input}}$$

Spatial Domain

$$\text{DFT}^{-1}(S = P F)$$

Frequency Domain

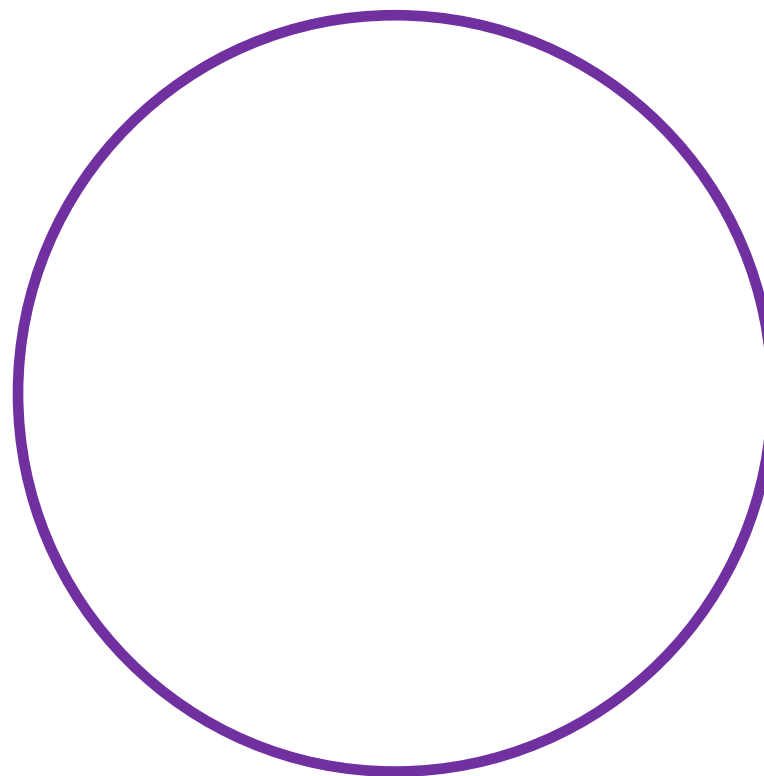
Remember the last lecture!

A Basic Reminder

A (very!) basic reminder to make this whole lecture easier to follow.

□ **Equation of a circle:**

$$x^2 + y^2 = r^2$$



See if you can spot how many times this equation turns up (in some form) in this lecture?

Low Pass Filtering

Operation: suppress high frequency components from the image.

The **ideal low pass filter** sets frequencies above a certain threshold to zero.

$$H(k_x, k_y) = \begin{cases} 1, & \sqrt{k_x^2 + k_y^2} \leq K \\ 0, & \sqrt{k_x^2 + k_y^2} > K \end{cases}$$

K is the cut-off distance (radius) from the Fourier image origin.

We multiply the Fourier image $F()$ by $H(k_x, k_y)$ to find the filtered Fourier image

$$S(k_x, k_y) = H(k_x, k_y) F(k_x, k_y)$$

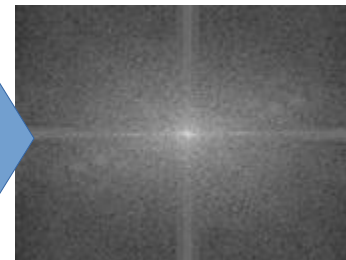
Uses: noise removal / image smoothing.

Ideal Low Pass Filter



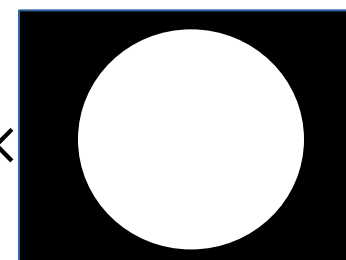
Original

DFT()



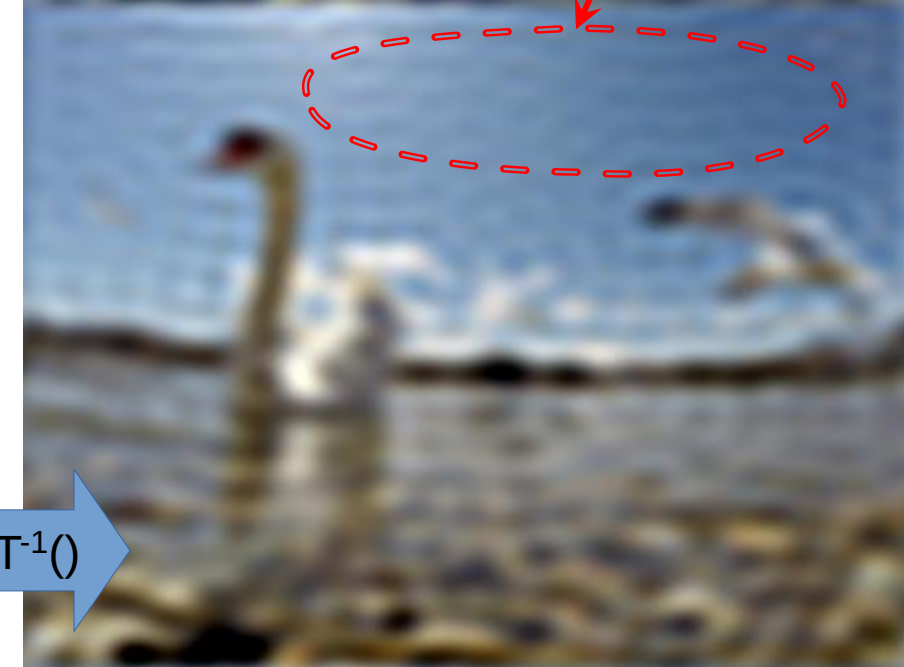
$F(k_x, k_y)$

$\times$



$H(k_x, k_y)$

DFT⁻¹()

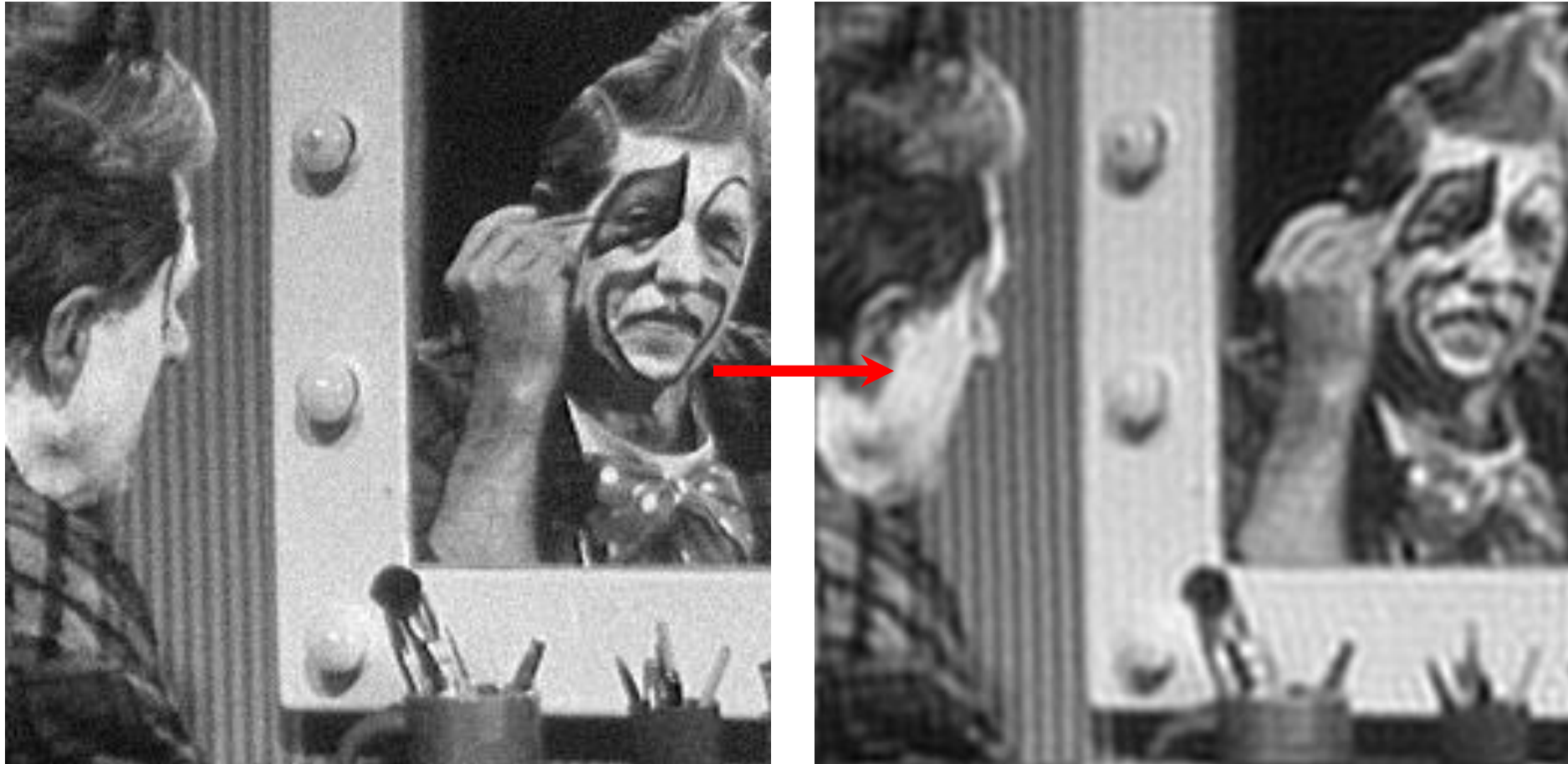


Remaining low frequencies

Noise: Ringing Effect

$$H(k_x, k_y) = \begin{cases} 1, & \sqrt{k_x^2 + k_y^2} \leq K \\ 0, & \sqrt{k_x^2 + k_y^2} > K \end{cases}$$

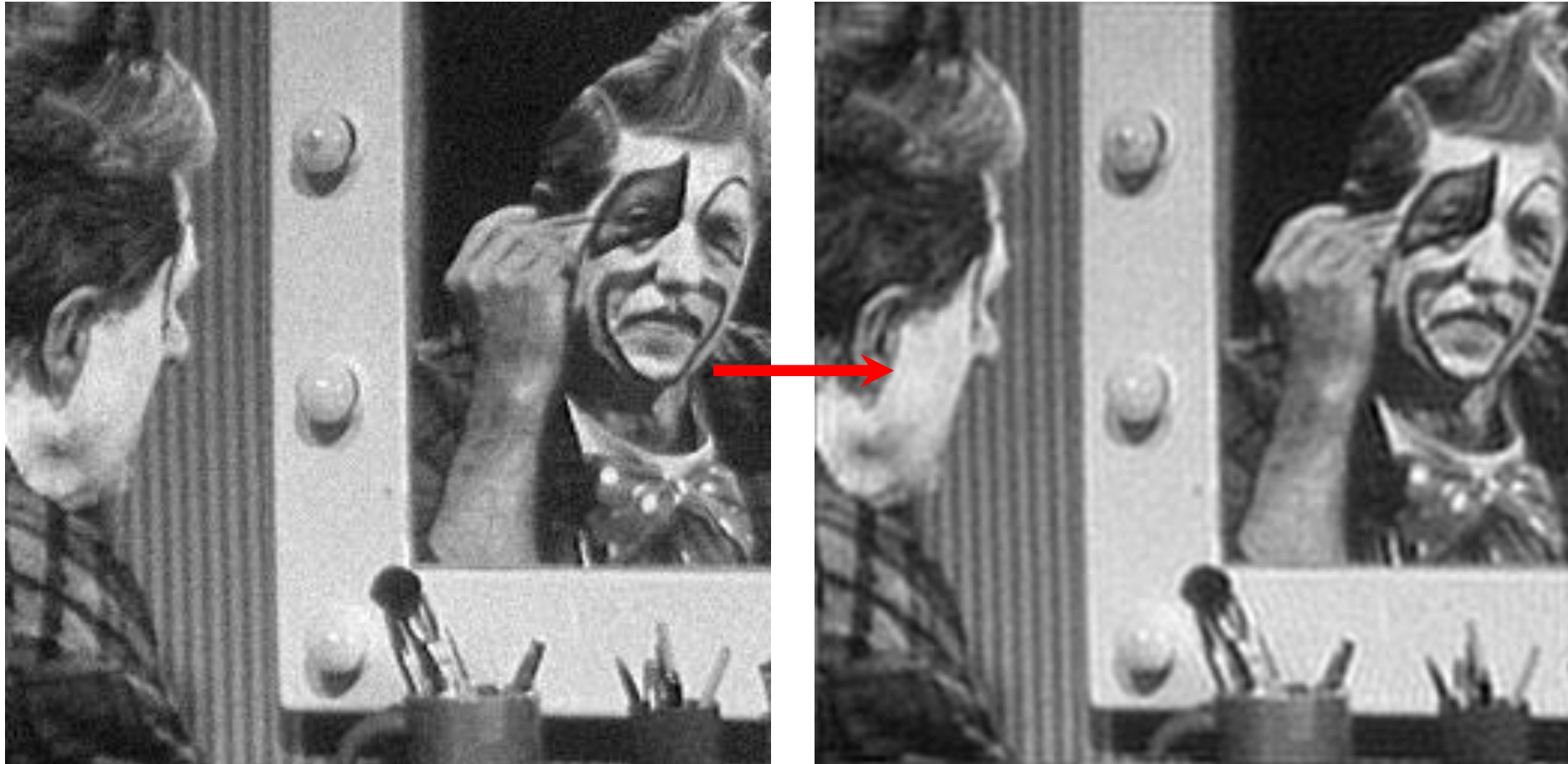
Ideal Low Pass Filter



Example: $K = 0.33$

- ❑ Ringing present in the image. High frequency detail lost (e.g. face).

Ideal Low Pass Filter



Example: $K = 0.5$

- ❑ Ringing still present in the image.

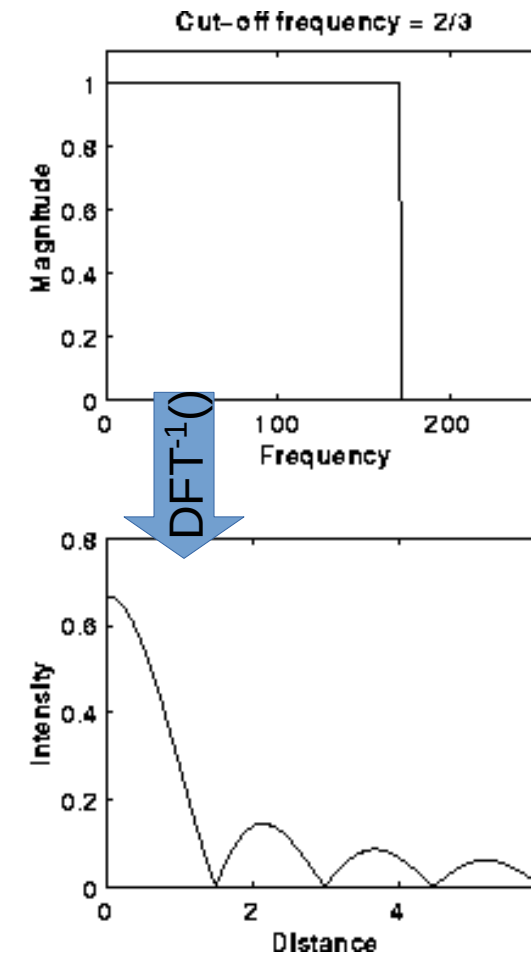
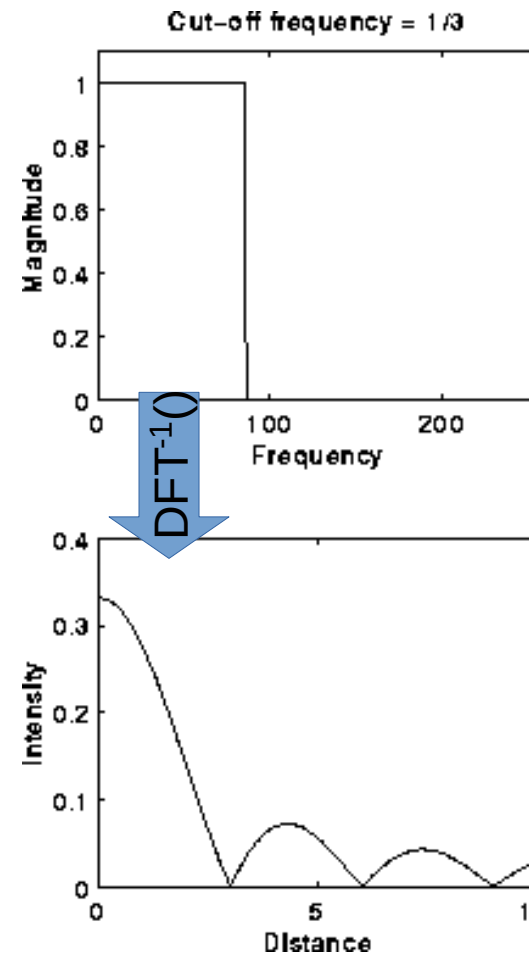
Example



Ringing in Ideal Low Pass Filter

- “Ringing” occurs at sharp edges in the image (illustrated here in 1D) as sharp frequency cut-offs are poorly approximated in the spatial domain.

- Gibbs effect (beyond scope)



If we have this in the Fourier domain.

We get this in spatial domain.

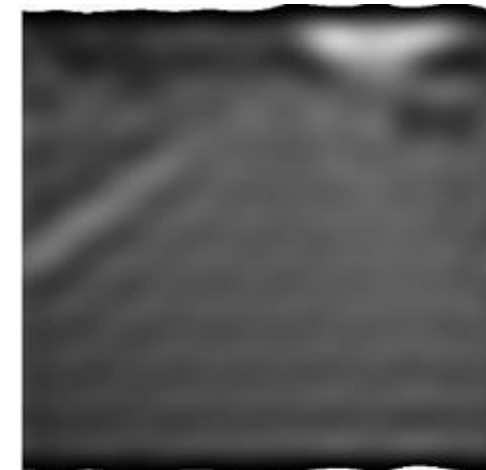
Ideal Low Pass Filter

- Smoothed the image but with ringing due to sharp frequency cut-off.

Here K_c measured in pixel distance from centre.



Original Image



$k_c = 10$



$k_c = 25$



$k_c = 50$



$k_c = 100$

Problem: Ringing in ideal low pass filter.

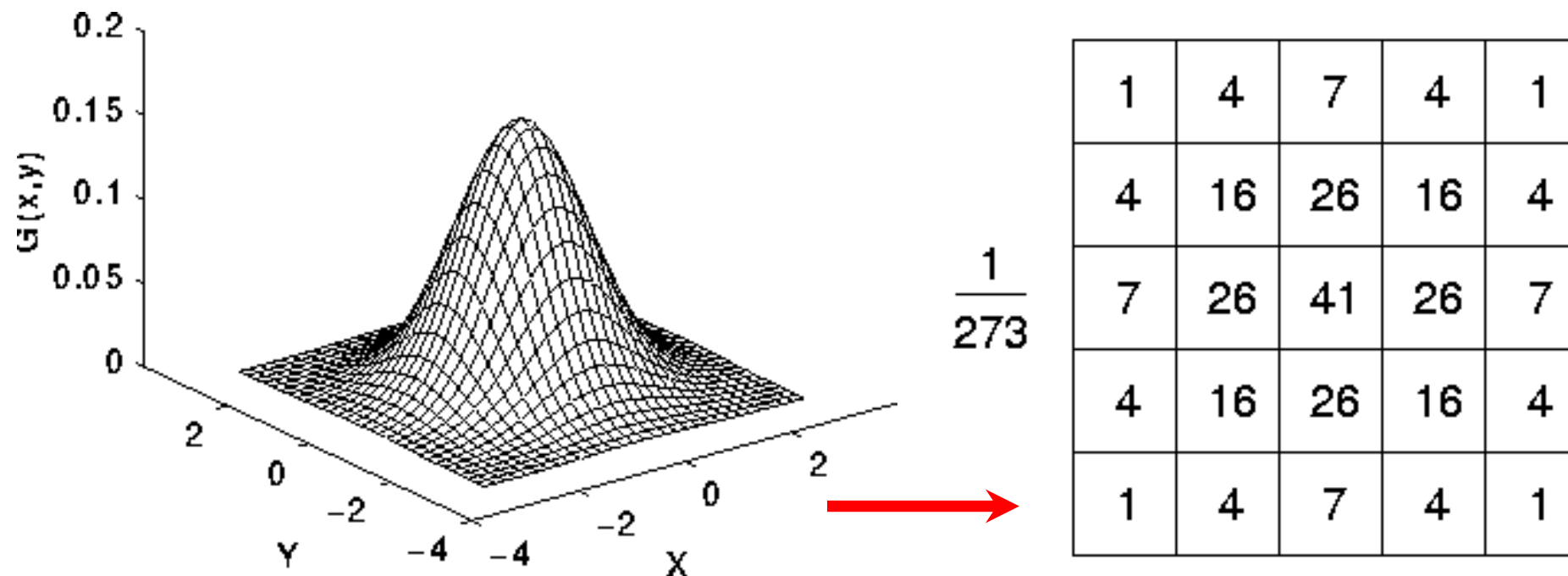
Solution: *low pass filters which are piecewise continuous in the Fourier domain*
- i.e. have no sharp edges.

Gaussian Low Pass Filter

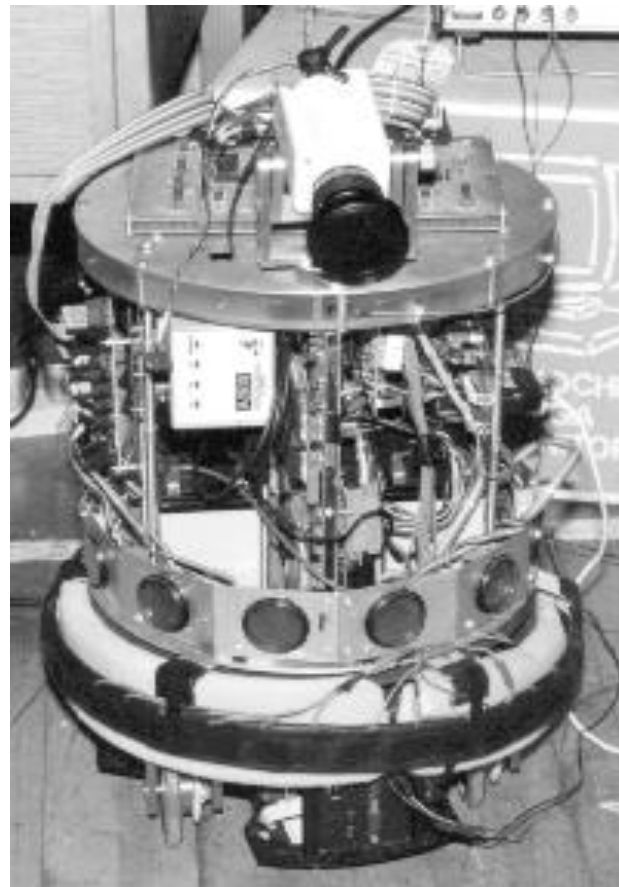
$$G(k_x, k_y) = \exp[-(k_x^2 + k_y^2)/2\sigma^2]$$

σ is the width of the filter at $1/e$.

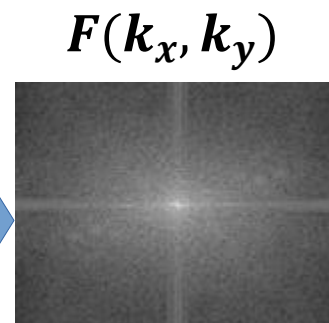
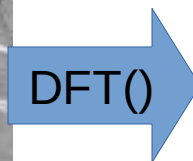
σ controls the bandwidth of this filter i.e. the range of frequencies allowed through.



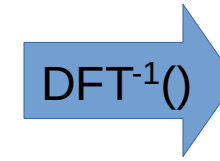
Gaussian Low Pass Filter



Original



×

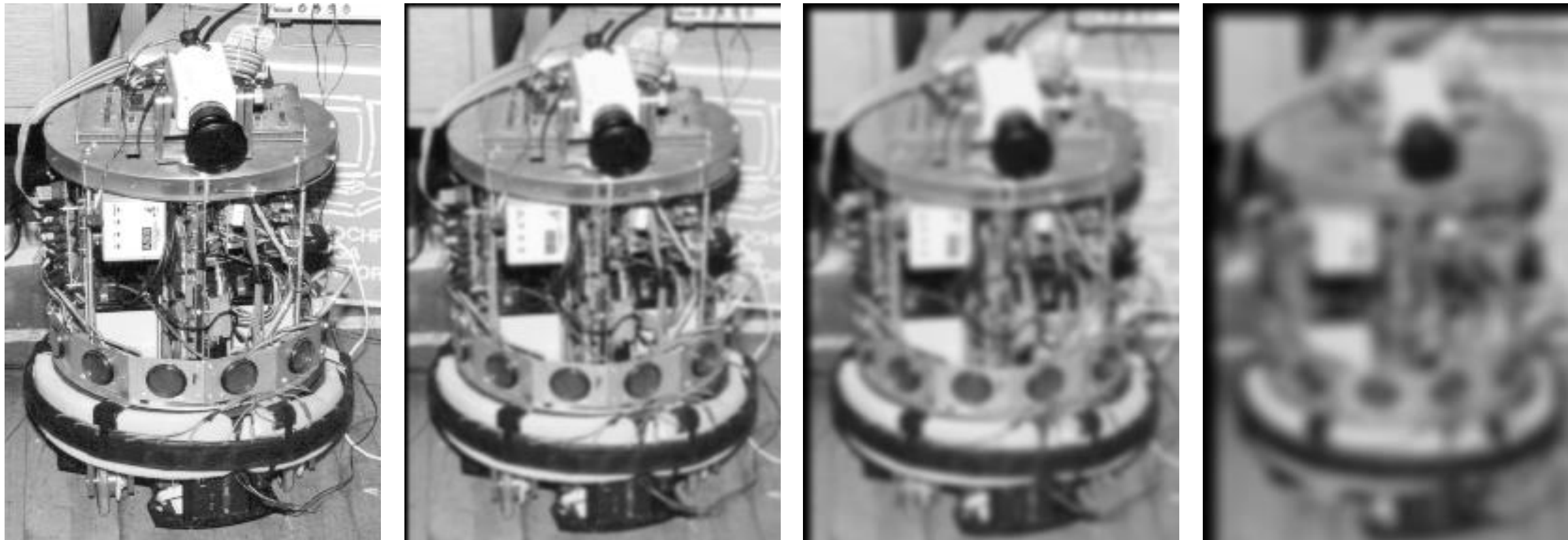


Remaining low frequencies

$$G(k_x, k_y) = \exp[-(k_x^2 + k_y^2)/2\sigma^2]$$

Gaussian Low Pass Filter

source: HIPR2 © 2003/4



Example: original and smoothed images with $\sigma = 1, 2, 4$ (left to right).

(used for Gaussian / Salt n' pepper noise removal)

- ❑ same effect as Gaussian spatial linear filtering (convolution, lectures 4/5).
 - **[convolution theorem]** - Fourier transform of a Gaussian is a Gaussian.

*Alternative
To Gaussian!*

Butterworth Low Pass Filter

Operation: smooth approximation to ideal low pass filter to avoid ringing effect via a **piecewise continuous radially symmetric filter**:

$$B(k_x, k_y) = \frac{1}{1 + \left(\frac{\sqrt{k_x^2 + k_y^2}}{K}\right)^{2n}}$$

- K = cut-off value (as before)
- n = user-defined positive integer called the **order of the filter**.

As n increases, the filter approaches the ideal high pass filter with cut-off value K .

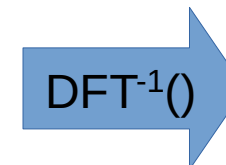
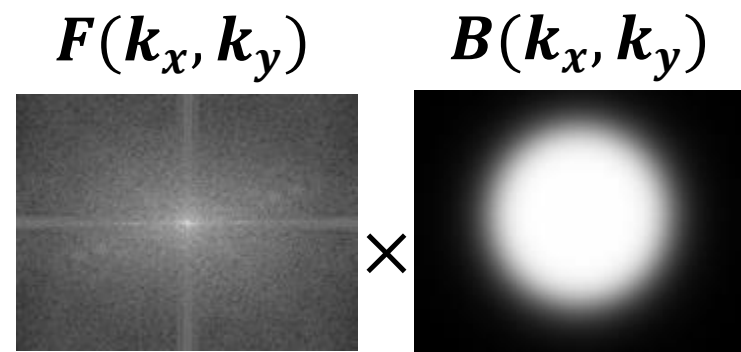
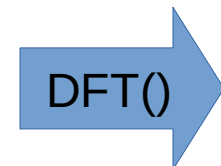
- but the computational cost similarly increases!!!

As n decreases, the filter becomes smoother and smoother.

Butterworth Low Pass Filter



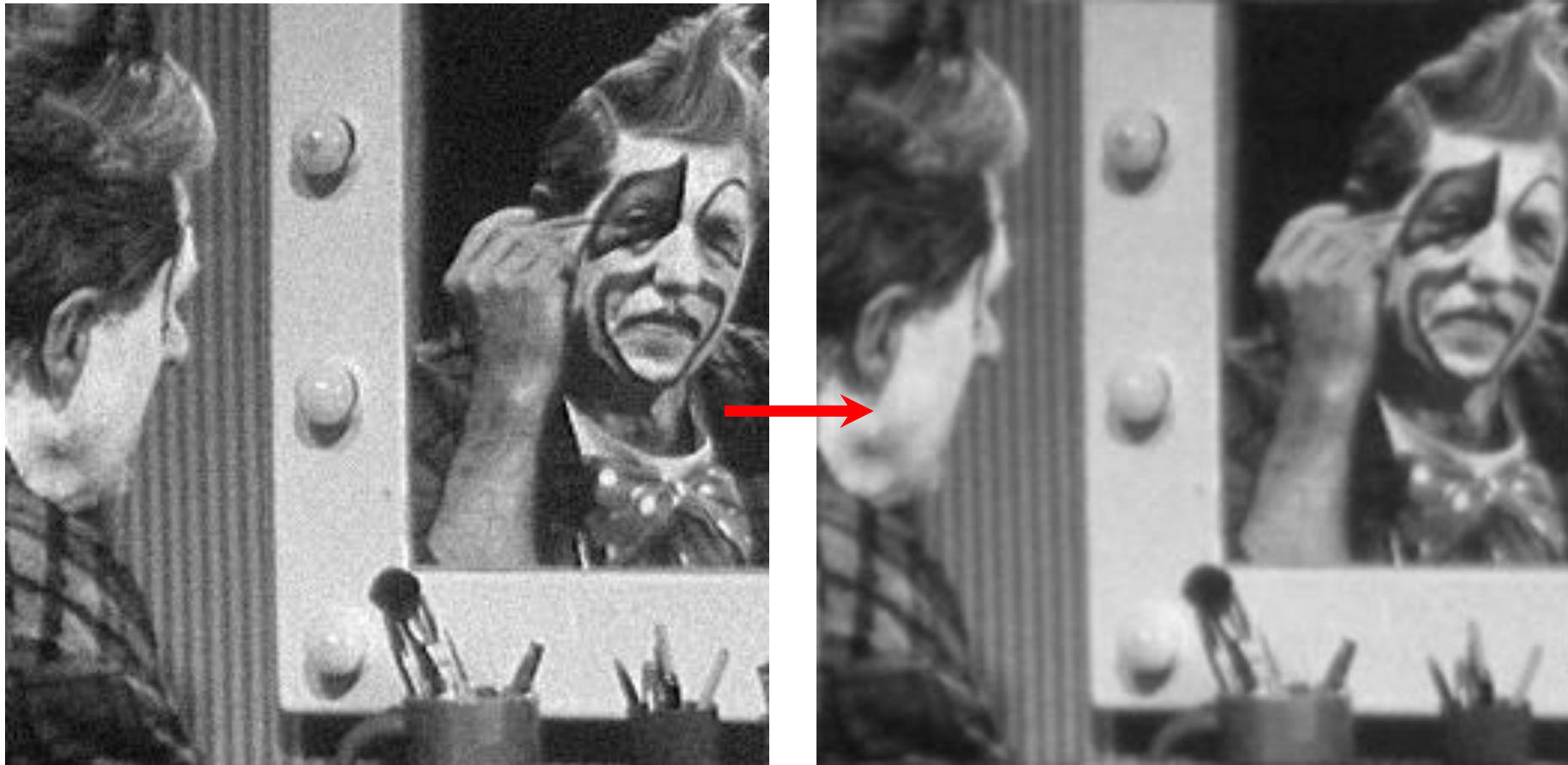
Original



Remaining low frequencies

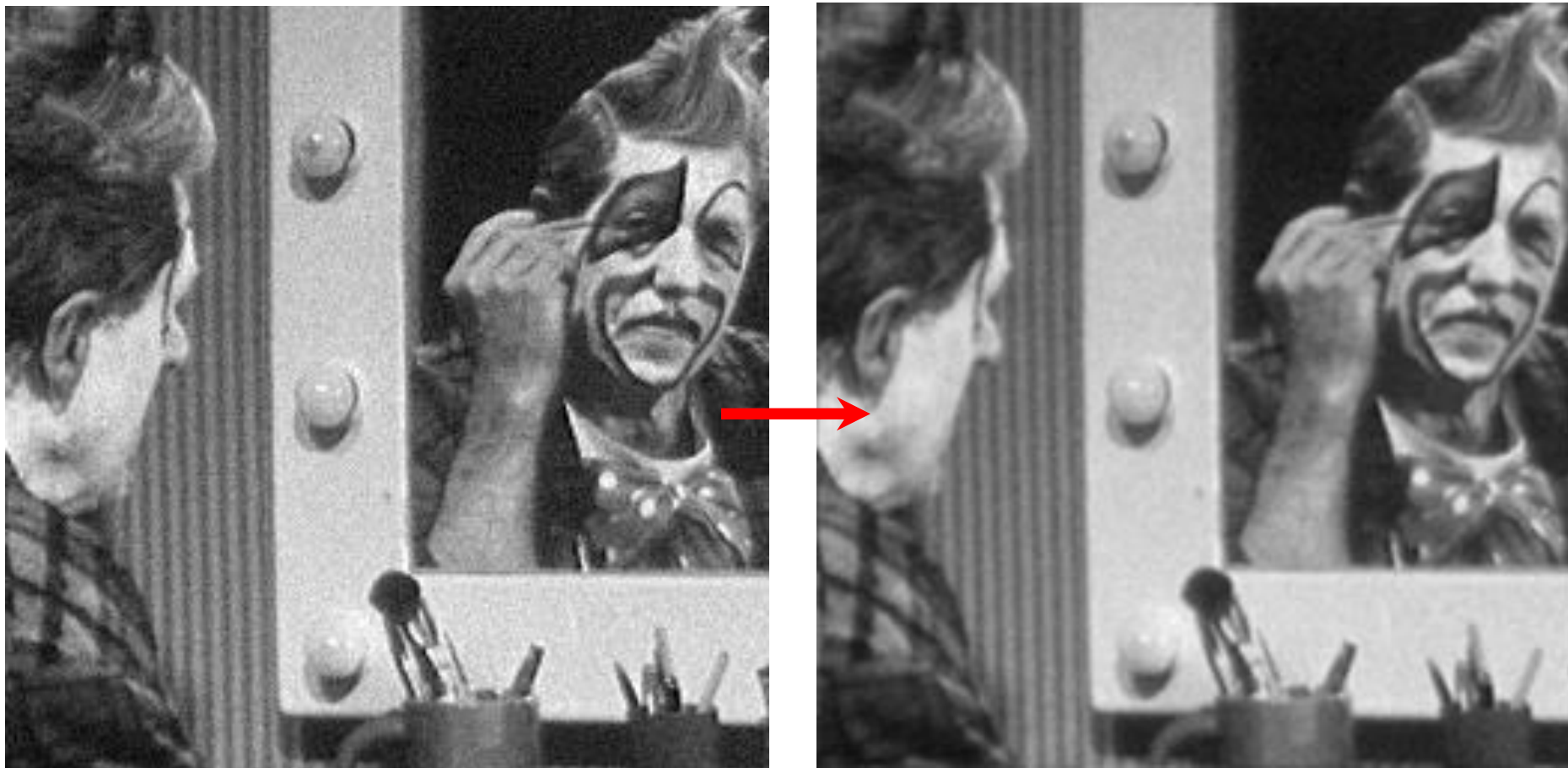
$$B(k_x, k_y) = \frac{1}{1 + \left(\frac{\sqrt{k_x^2 + k_y^2}}{K} \right)^{2n}}$$

Butterworth Low Pass Filter



- **Example:** $K = 0.33$
 - No ringing but some loss of detail.

Butterworth Low Pass Filter



□ **Example:** $K = 0.5$

- No ringing, less loss of detail, but more noise remains.
- We have a detail / noise trade-off.

Butterworth Low Pass Filter

- Smoothed the images with no ringing due to the gradual frequency cut-off.

Here, K_c is measured in pixel distance from centre.



$K_c = 10$



$K_c = 25$



$K_c = 50$



$K_c = 100$

... *there is also* “High Pass Filtering”

High Pass Filtering

Operation: suppress low frequency components from the image.

The **ideal high pass filter** sets frequencies below a certain threshold to zero

$$H(k_x, k_y) = \begin{cases} 0, & \sqrt{k_x^2 + k_y^2} \leq K \\ 1, & \sqrt{k_x^2 + k_y^2} > K \end{cases}$$

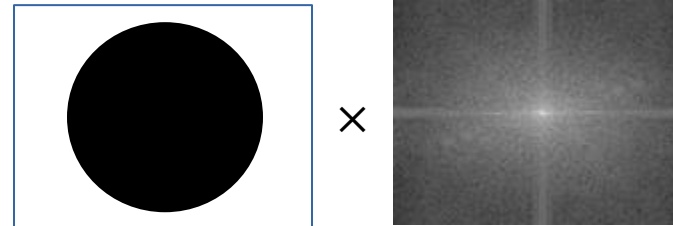
K is the cut-off distance (radius) from the Fourier image origin.

We multiply the Fourier image $F()$ by $H(k_x, k_y)$ to find the filtered Fourier image

$$S(k_x, k_y) = H(k_x, k_y) F(k_x, k_y)$$

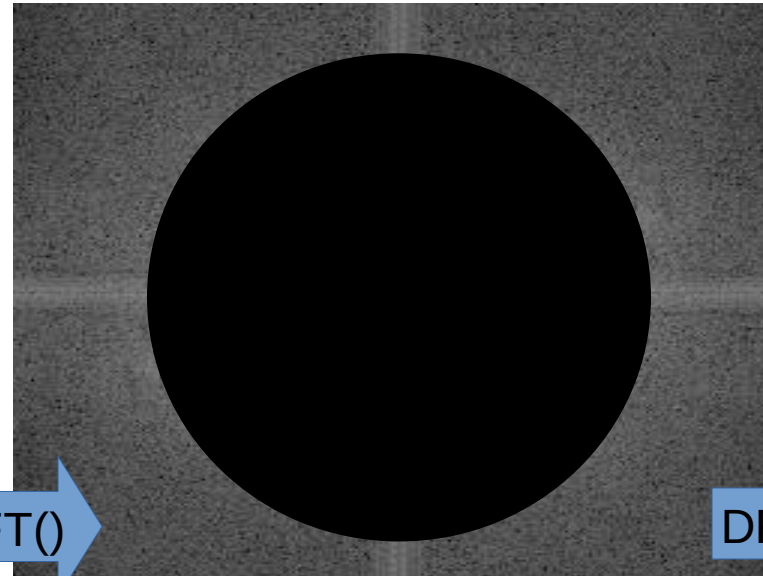
Uses: detect edges / edge enhancement.

High Pass Filter

 $H(k_x, k_y)$
 $F(k_x, k_y)$


Original

DFT()



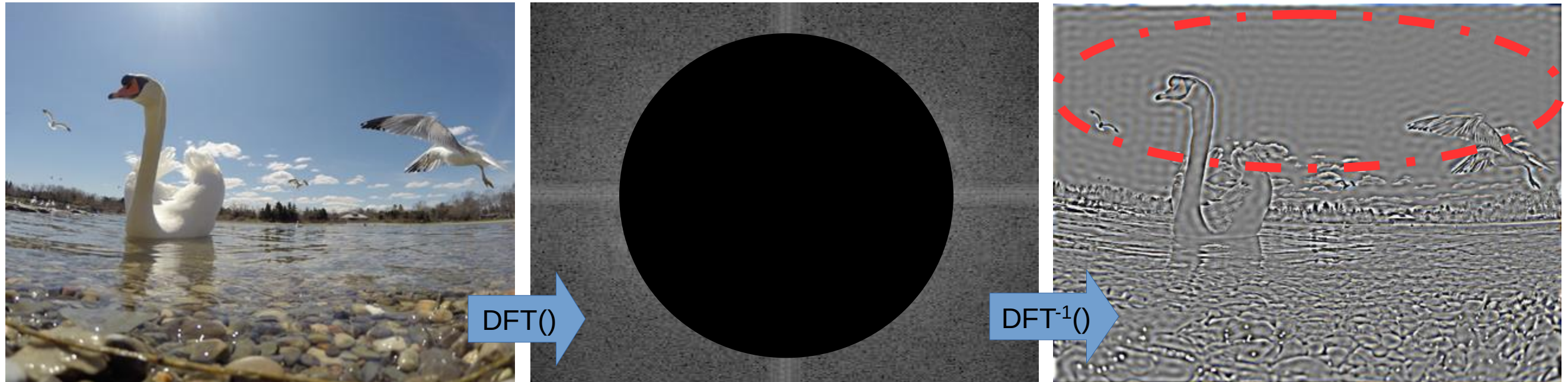
DFT⁻¹()



Remaining high frequencies

$$H(k_x, k_y) = \begin{cases} 0, & \sqrt{k_x^2 + k_y^2} \leq K \\ 1, & \sqrt{k_x^2 + k_y^2} > K \end{cases}$$

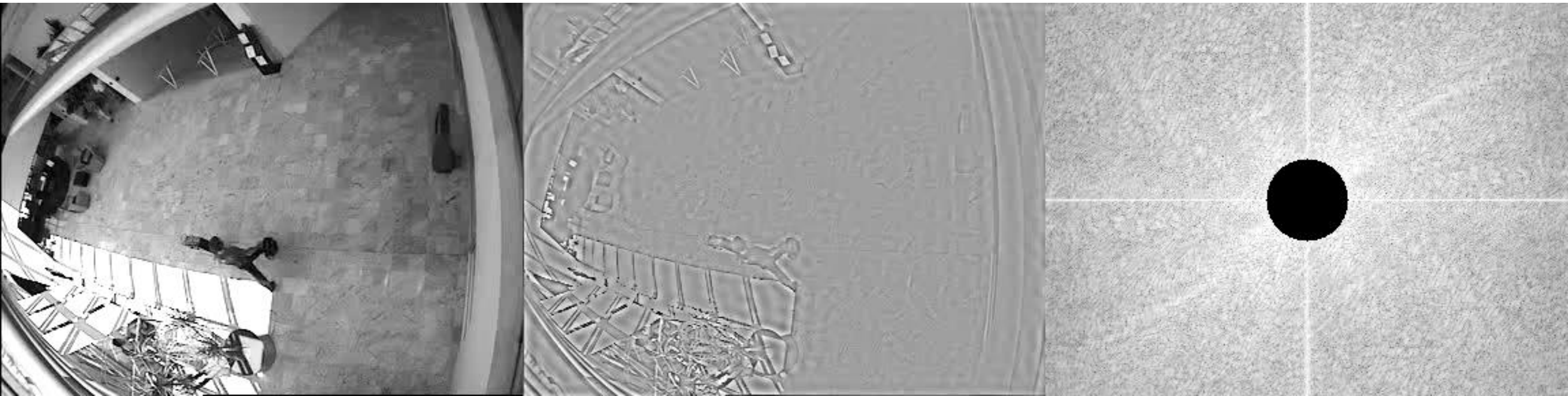
Strange Effects ...



- ❑ We also observe high frequencies where they do not exist in the corresponding spatial images.

The **ringing artefacts** (i.e. processing noise) are introduced by the sharp $\{0 \rightarrow 1\}$ cut-off of frequencies in the Fourier domain.

Example



Butterworth High Pass Filter

Operation: smooth approximation to the high pass filter to avoid ringing effects.

A piecewise continuous radially symmetric filter:
$$B(k_x, k_y) = \frac{1}{1 + \left(\frac{K}{\sqrt{k_x^2 + k_y^2}} \right)^{2n}}$$

- K = cut-off value (as before)
- n = user-defined positive integer called the **order of the filter**.

As n increases, the filter approaches the ideal high pass filter with cut-off value K .

- but the computational cost similarly increases!!!

As n decreases, the filter will contain a smoother and smoother transition.

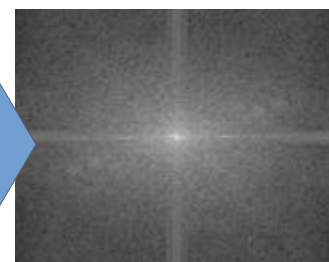
Butterworth High Pass Filter



Original

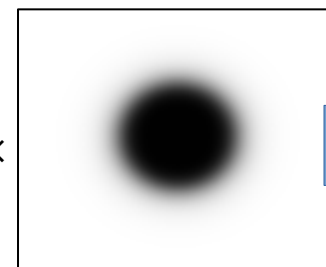
DFT()

$F(k_x, k_y)$



×

$B(k_x, k_y)$



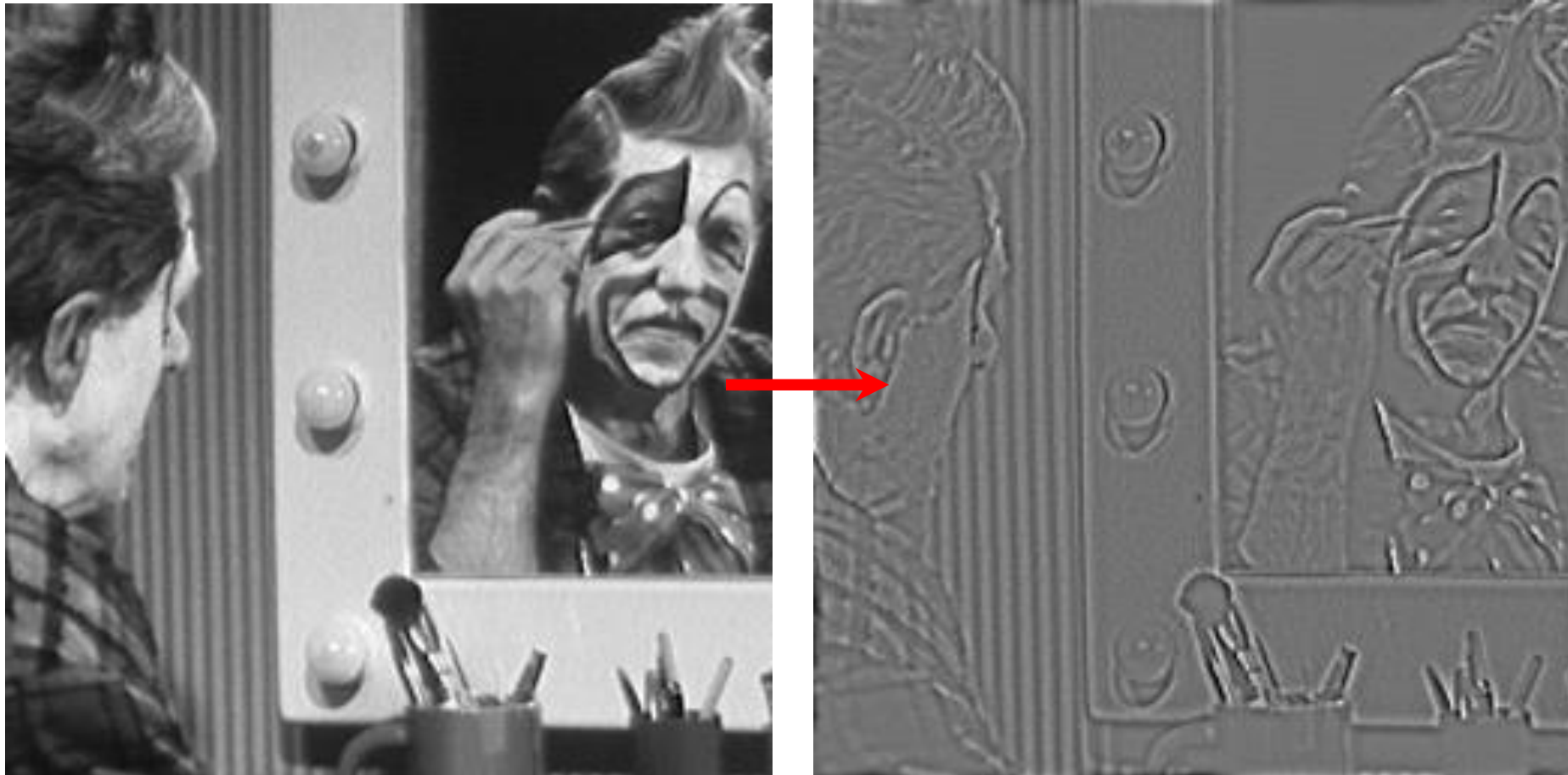
DFT⁻¹()



Remaining high frequencies

$$B(k_x, k_y) = \frac{1}{1 + \left(\frac{K}{\sqrt{k_x^2 + k_y^2}} \right)^{2n}}$$

Butterworth High Pass Filter

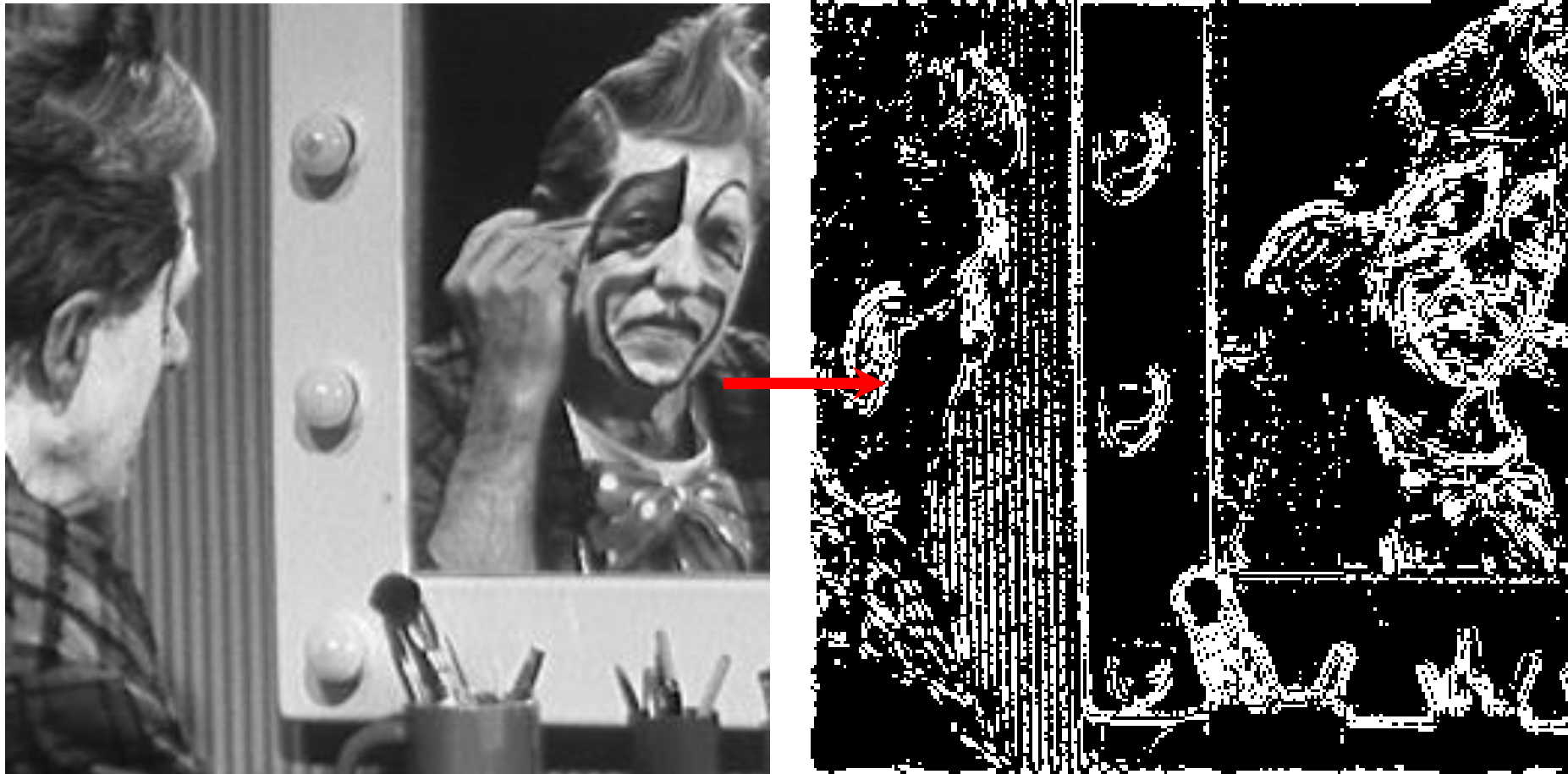


Example: $K = 0.5$ (measured as normalized distance $0 \rightarrow 1$ from centre to edge).

The result has both $+ve/-ve$ output. For display, a constant is added to pixels.

- Gives embossed effect as shown.

Butterworth High Pass Filter



source: HIPR2 © 2003/4

Example: Simple edge detection

- $K = 0.5$, take absolute value of filter result and apply thresholding.

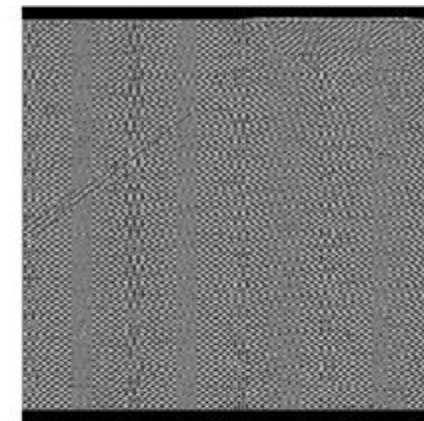
Butterworth High Pass Filter

cut-off = K_c

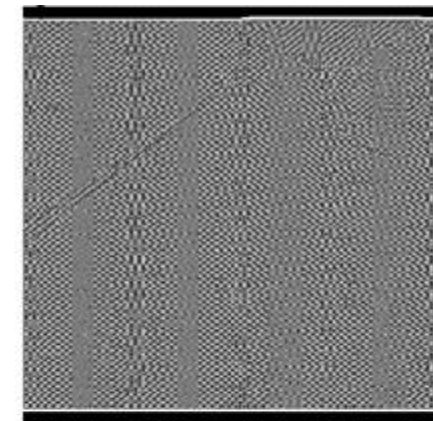
- Used to isolate upper frequency components in image.
- Here K_c measured in pixel distance from centre.



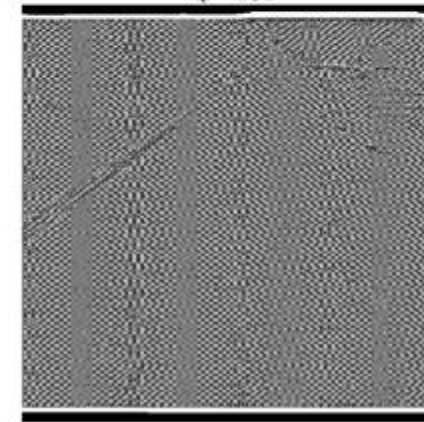
Original Image



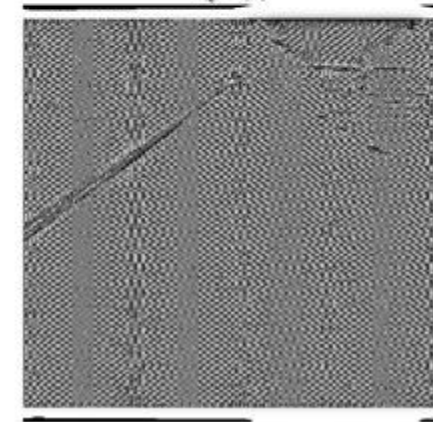
$k_c = 181$



$k_c = 90$



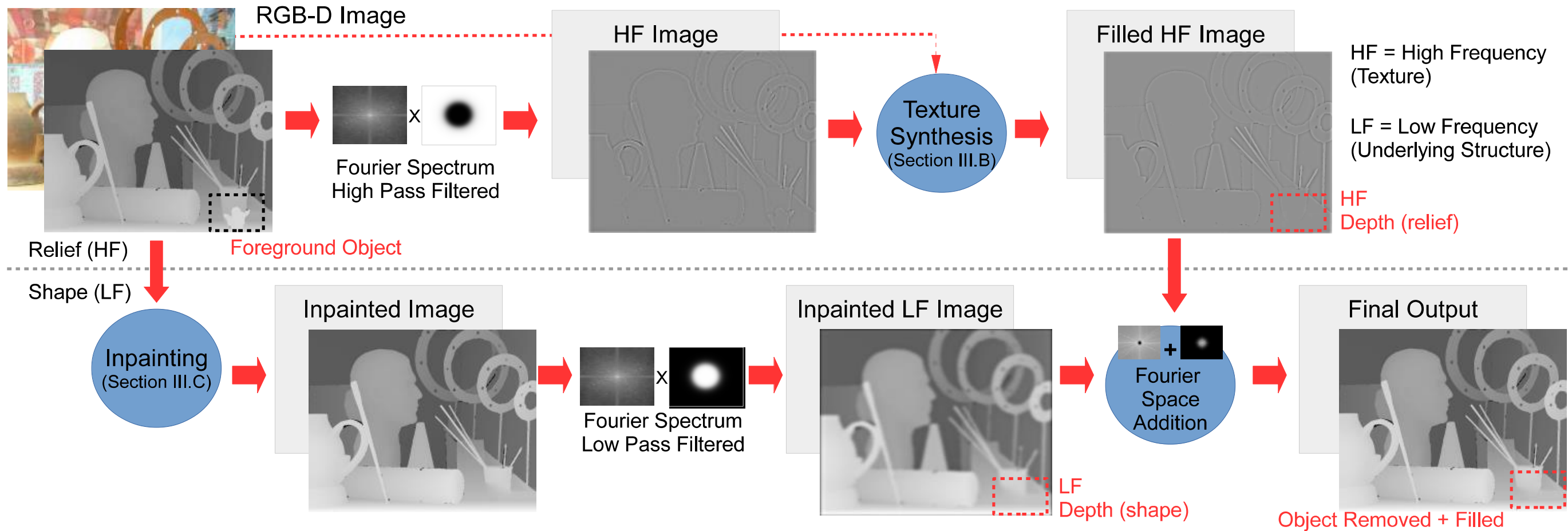
$k_c = 45$



$k_c = 22$

Butterworth Filter - Application

Back to Butterworth - A Fourier Basis for 3D Surface Relief Hole Filling within RGB-D Imagery



[Atapour et al., *ICPR*, 2016]

Butterworth Filter - Application

Back to Butterworth - A Fourier Basis for 3D Surface Relief Hole Filling within RGB-D Imagery



[Atapour et al., *ICPR*, 2016]

Butterworth Filter - Application

Back to Butterworth - A Fourier Basis for 3D Surface Relief Hole Filling within RGB-D Imagery



[Atapour et al., *ICPR*, 2016]

Quick Demonstration



- Simple High/Low Pass Filtering:

<https://github.com/atapour/ip-python-opencv/blob/main/low-high-pass-filter-fourier.py>

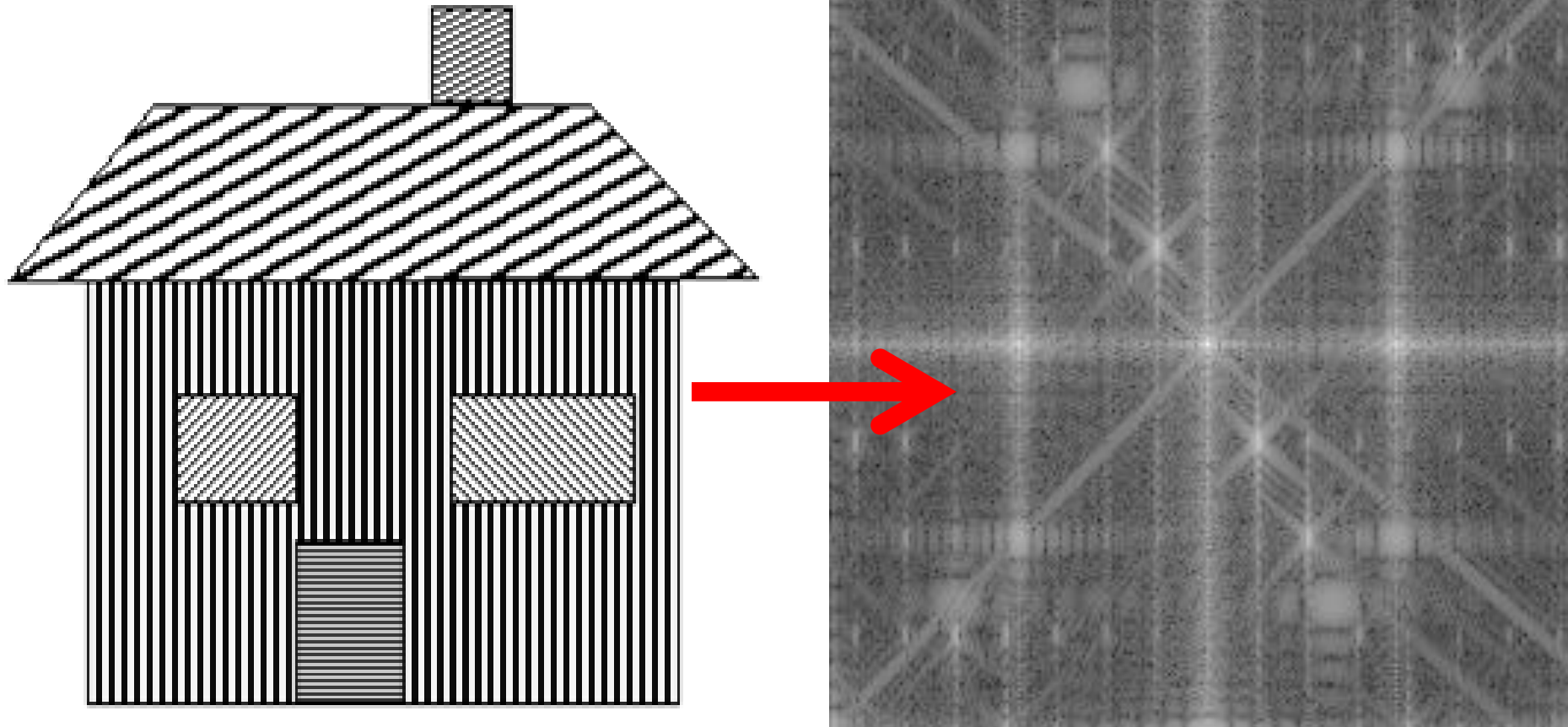


- Butterworth High/Low Pass Filtering:

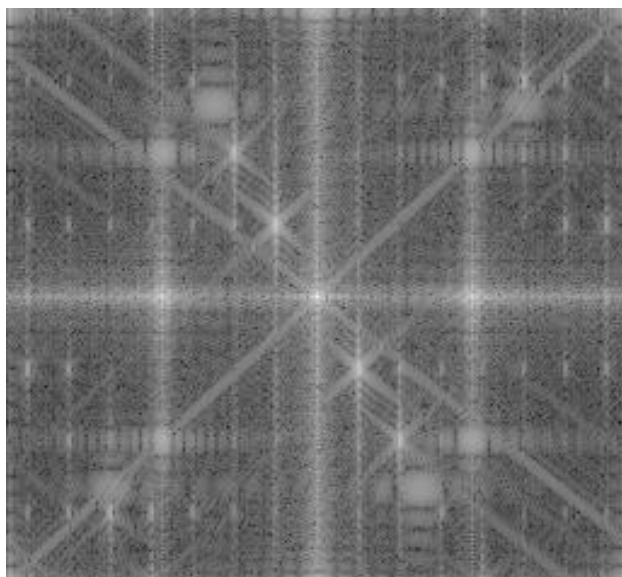
<https://github.com/atapour/ip-python-opencv/blob/main/butterworth-low-high-pass-filter.py>

Generalisation : Band Pass Filtering

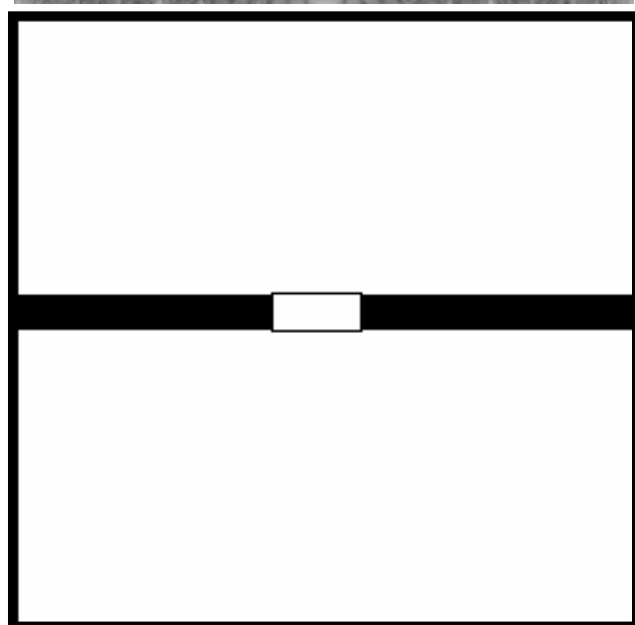
- ❑ **Operation:** a filter with **two or more frequency thresholds** that passes frequencies within a given range.
 - *e.g.* striped house:



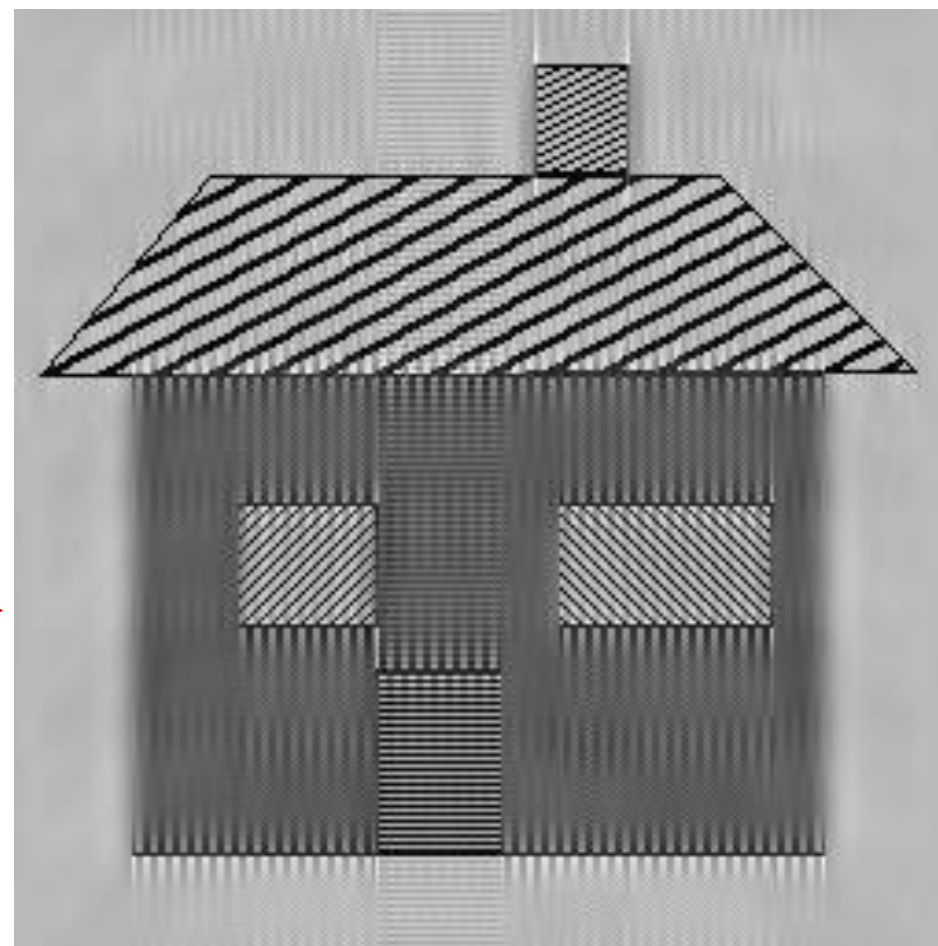
Example : Band Pass Filtering



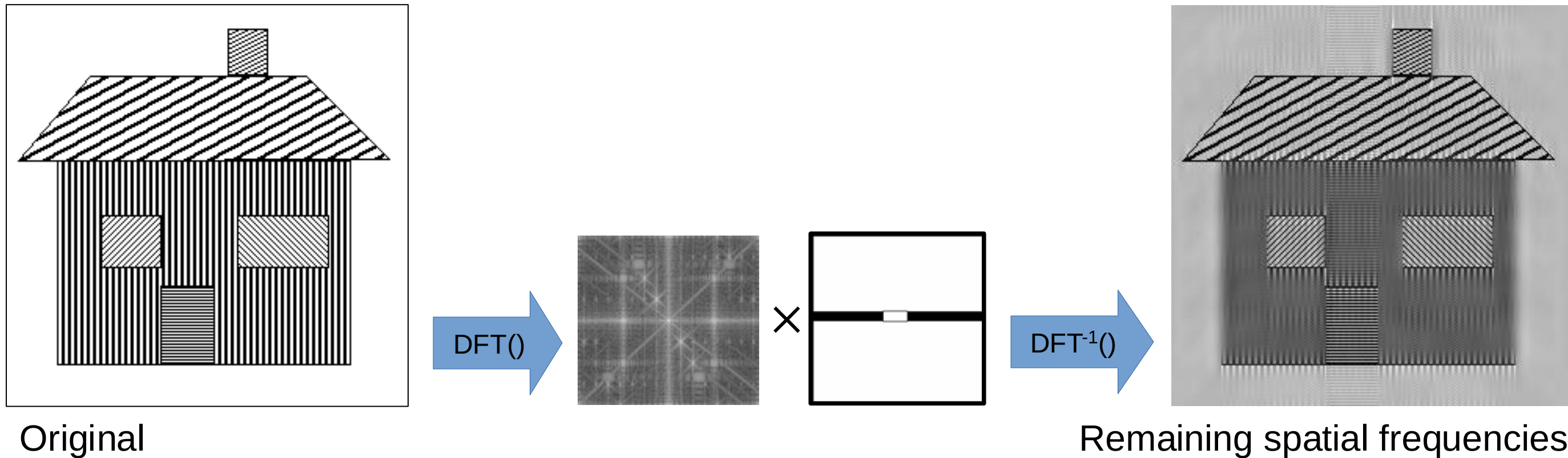
- Use following mask (white = 1, black = 0) to remove vertical stripes



source: HIPR2 © 2003/4



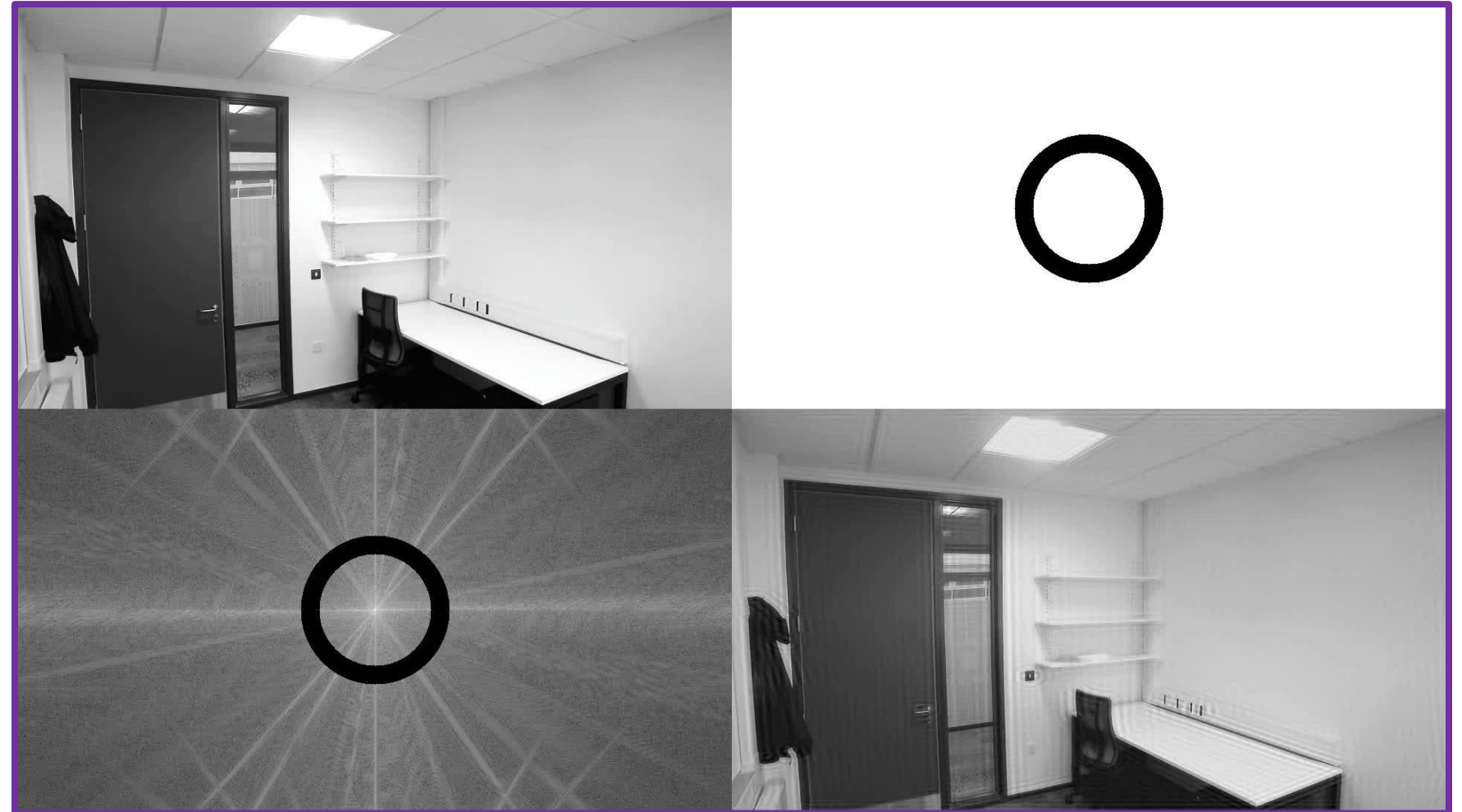
Example : Band Pass Filtering



□ Only frequencies in areas where mask is > 0 are retained.

Example : Band Pass Filtering

- ❑ Maintain a zone of frequencies.

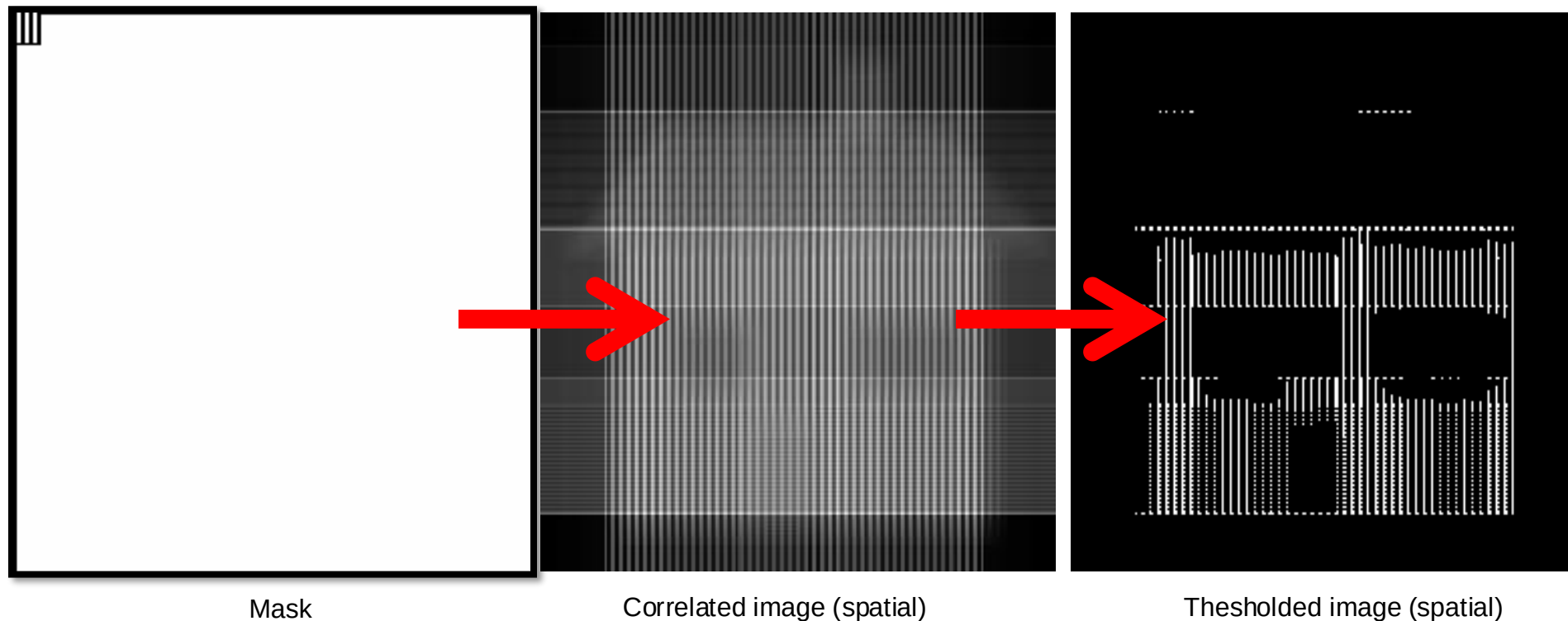


How could we
create such a filter?

<https://youtu.be/iPnSDdnIC8k>

Fourier Space Correlation

- ❑ **Operation:** find all the regions with certain properties or patterns
- ❑ **Process (in Fourier):** transform image and mask into Fourier space and multiply them together.
 - Frequencies in mask remain (amplified), whilst others set zero (attenuated).



e.g. as per above identify all the vertically striped regions

2D Cross Correlation

[basic image template matching]

- ❑ **Correlation** in Image Processing is **very similar to convolution**
 - **key difference:** kernel, p , is not necessarily symmetric
- ❑ **Correlation** is primarily used for **matching**.
 - *e.g.* template matching.
 - kernel represents a template (e.g. blob, line orientation,...)
 - response of kernel relates to how well that kernel matches at given position, *i.e.* pixel difference in spatial domain:

$$I_{\text{output}}(i, j) = \sum_{k=-N/2}^{N/2} \sum_{l=-M/2}^{M/2} \|I_{\text{input}}(i + k, j + l) - m_{kl}\|$$

- often squared difference is used (i.e. “sum of squared distances”).

Cross-Correlation

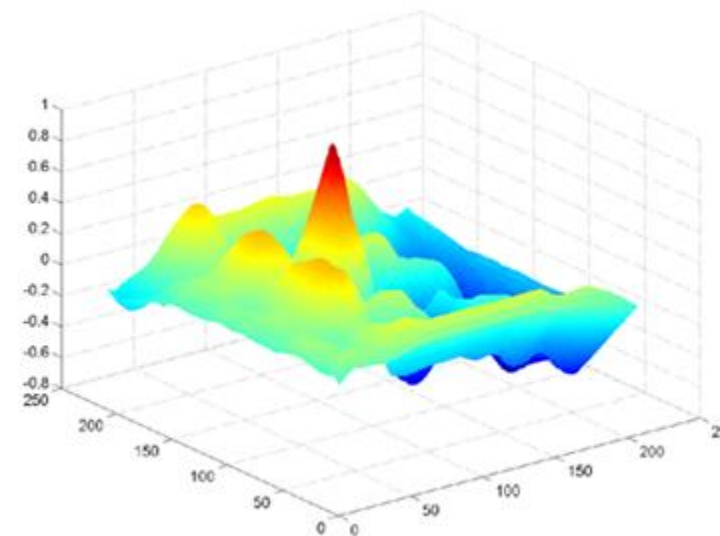
I_{input}



Mask, m



?



- Cross-correlation response (plotted in 2D) and normalised ($0 \rightarrow 1$) so strong matches for the mask are high.

Source: <http://uk.mathworks.com/help/images/examples/registering-an-image-using-normalized-cross-correlation.html>

Example: Correlation



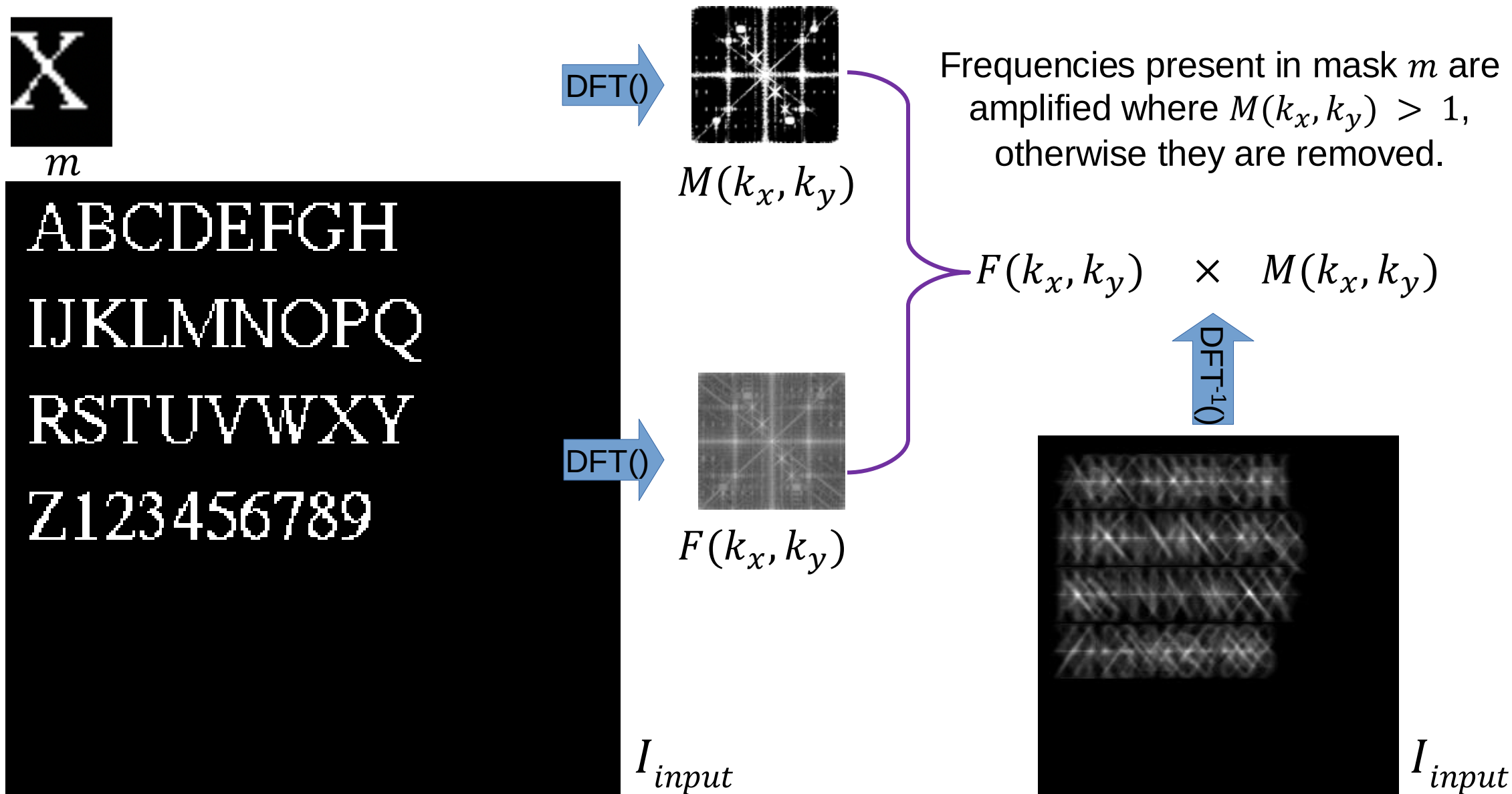
m

ABCDEFGH
 IJKLMNOPQ
 RSTUVWXY
 Z123456789

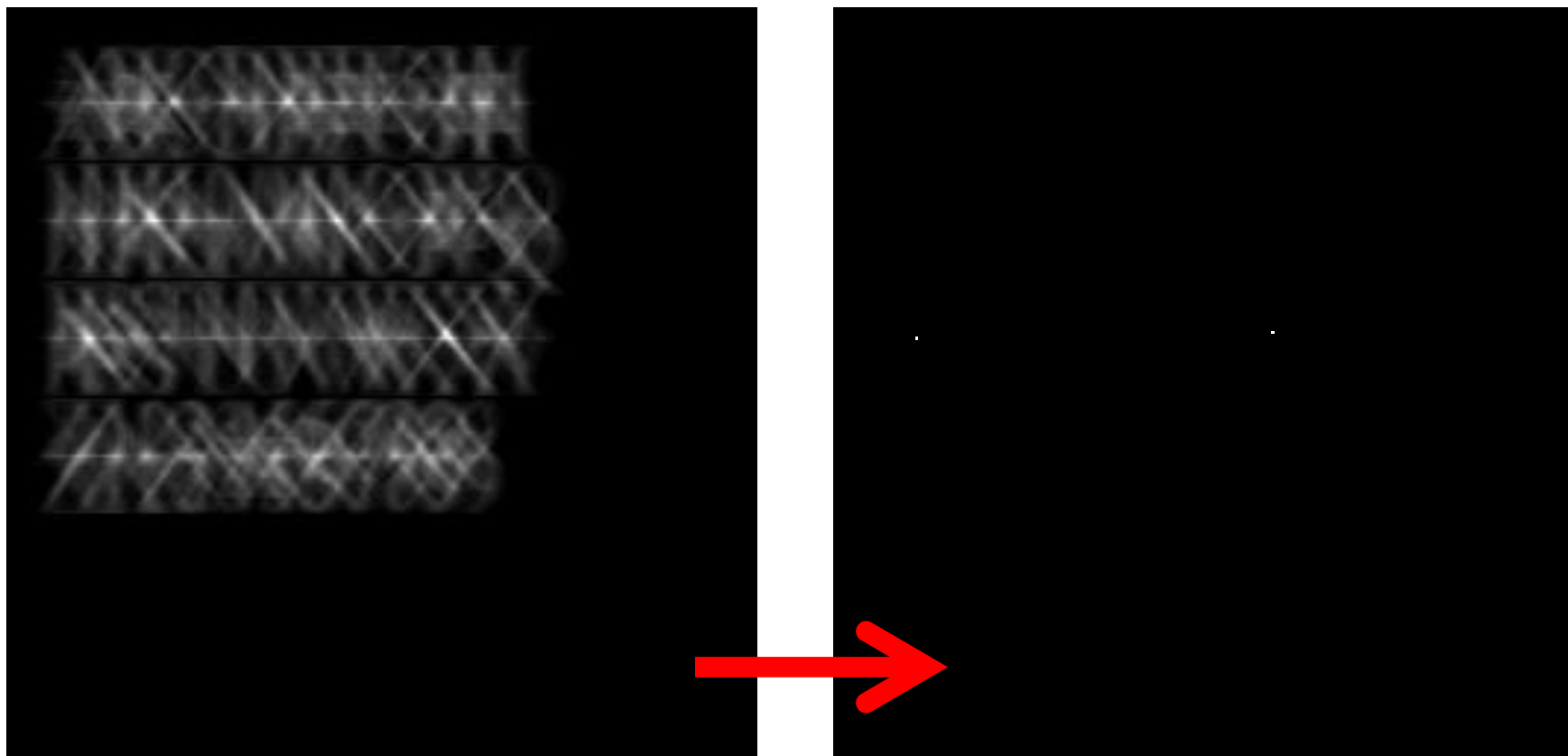
□ Where does an “X” pattern occur ?

I_{input}

Example: Correlation via DFT



Example: Correlation via DFT

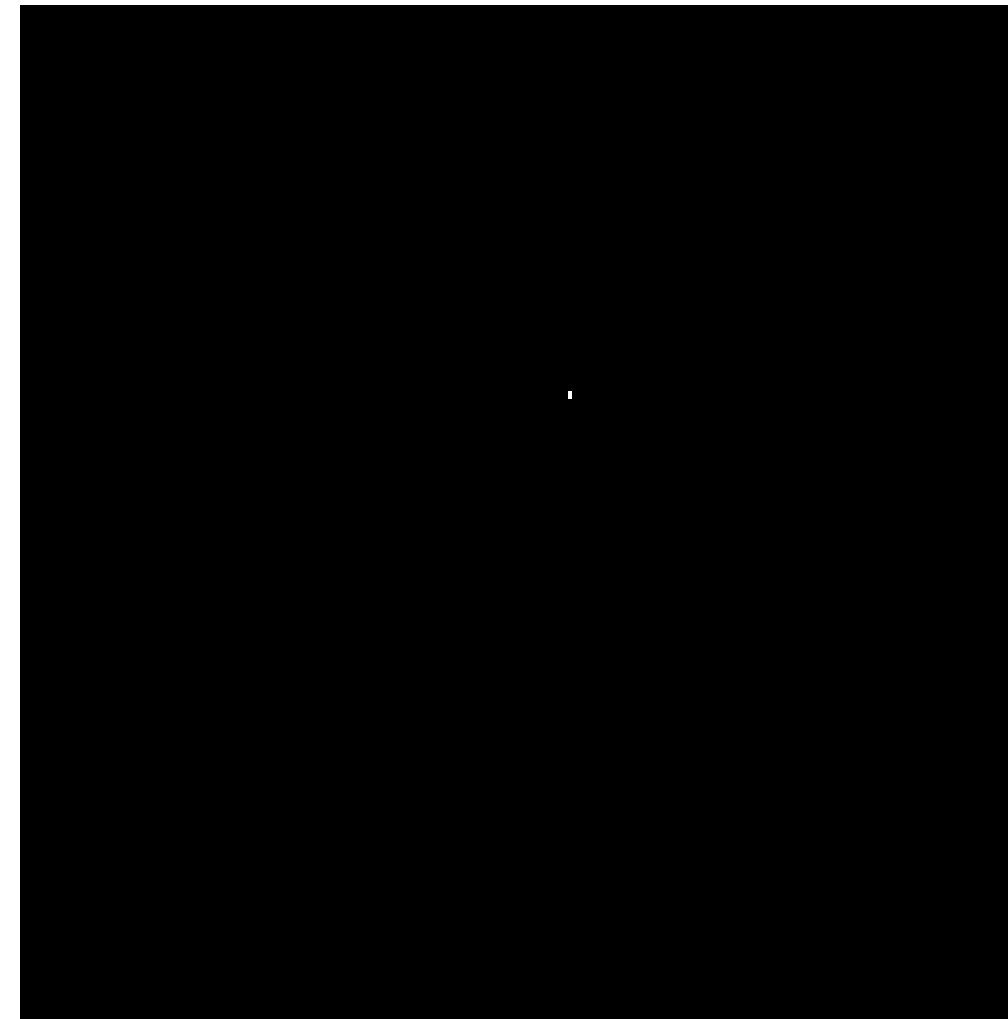
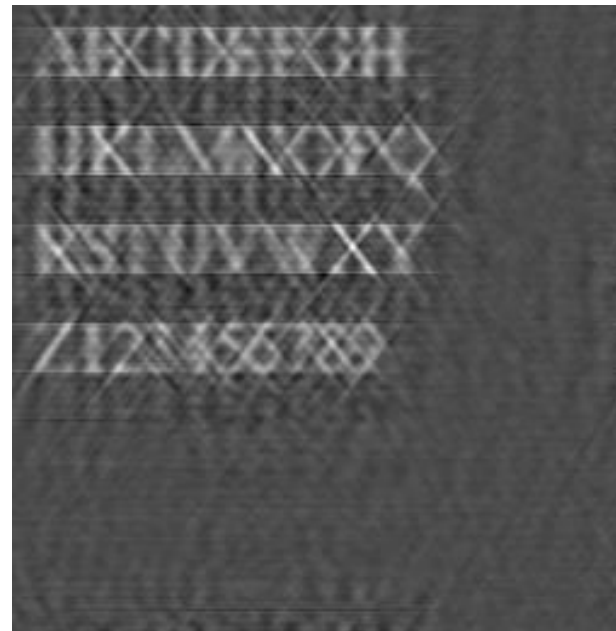
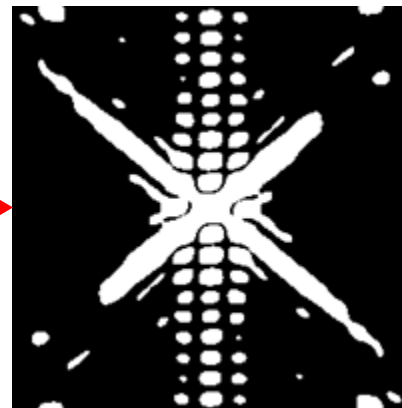


- ❑ Spatial domain result of Fourier correlation + thresholded:
- ❑ One false positive pattern match for X (letter R!)
 - Also, *not rotation or scale invariant* (***not generalised pattern matching!***)

Example: Correlation via DFT

Alternative:

- threshold Fourier image of mask (i.e. X):
 - Isolate most important frequencies in pattern.
 - Use this for correlation instead.
 - result: **one X!**



Quick Demonstration

- Correlation Template Matching:

https://github.com/atapour/ip-python-opencv/blob/main/correlation_template_matching.py



Fourier Deconvolution



- ❑ **De-convolution**: remove the effects of a given filter function.
- ❑ **Divide** the image by the filter mask in **Fourier Space** to perform deconvolution
 - Inverse of convolution, used as basis for **image deblurring**.

Summary

Fourier filtering

- ❑ Low pass filtering
- ❑ High pass filtering
- ❑ Ringing

Fourier space filters

- ❑ Ideal filters
- ❑ Butterworth
- ❑ Gaussian

Correlation

- ❑ Spatial domain concept
- ❑ Fourier domain approach (efficiency)

